

Scalable Payment Architecture with Stripe + Razorpay in Node.js (Class-Based)

This architecture is designed for:

- scalability
- modularity
- multiple payment providers
- subscriptions
- webhooks
- clean separation of concerns
- dependency injection
- provider abstraction

The goal is:

Business logic should NOT directly depend on Stripe or Razorpay SDKs.

High-Level Architecture

```
Routes
|
Controllers
|
Services
|
Payment Provider Abstraction
|
Stripe / Razorpay Providers
|
Repositories
|
Database
```

Folder Structure

```
src/
|
├── config/
|   └── payment.config.js
```

```
|
|
├─ controllers/
|   └─ payment.controller.js
|
├─ services/
|   └─ payment.service.js
|
├─ repositories/
|   ├── subscription.repository.js
|   └─ payment.repository.js
|
├─ providers/
|   └─ payment/
|       ├── payment.provider.js
|       ├── stripe.provider.js
|       └─ razorpay.provider.js
|
├─ factories/
|   └─ payment-provider.factory.js
|
├─ models/
|   └─ subscription.model.js
|
├─ webhooks/
|   ├── stripe.webhook.js
|   └─ razorpay.webhook.js
|
├─ routes/
|   └─ payment.routes.js
|
├─ app.js
|
└─ server.js
```

IMPORTANT ARCHITECTURE IDEA

Your app should think in BUSINESS TERMS:

```
Create Subscription
Cancel Subscription
Refund Payment
```

NOT:

```
Create Stripe Checkout Session
Create Razorpay Subscription Object
```

Provider-specific complexity stays inside provider classes.

1. Install Packages

```
npm install express mongoose dotenv stripe razorpay
```

2. Environment Variables

`.env`

```
PORT=3000

MONGO_URI=mongodb://127.0.0.1:27017/payments

STRIPE_SECRET_KEY=sk_test_xxx
STRIPE_WEBHOOK_SECRET=whsec_xxx

RAZORPAY_KEY_ID=rzp_test_xxx
RAZORPAY_SECRET=xxx
RAZORPAY_WEBHOOK_SECRET=xxx
```

3. Subscription Model

`models/subscription.model.js`

```
import mongoose from "mongoose";

const subscriptionSchema = new mongoose.Schema(
  {
    userId: {
      type: String,
      required: true,
    },

    provider: {
      type: String,
      enum: ["stripe", "razorpay"],
    },
  }
);
```

```
      required: true,
    },

    providerSubscriptionId: {
      type: String,
    },

    providerCustomerId: {
      type: String,
    },

    status: {
      type: String,
      default: "pending",
    },

    planType: {
      type: String,
    },

    billingInterval: {
      type: String,
    },

    currentPeriodEnd: {
      type: Date,
    },

    metadata: {
      type: Object,
      default: {},
    },
  },
  {
    timestamps: true,
  }
);

export const Subscription = mongoose.model(
  "Subscription",
  subscriptionSchema
);
```

WHY THIS MODEL IS IMPORTANT

This is YOUR internal business model.

NOT:

Stripe model
OR
Razorpay model

Both providers map into this common internal structure.

4. Repository Layer

repositories/subscription.repository.js

```
export class SubscriptionRepository {

  constructor(subscriptionModel) {
    this.subscriptionModel = subscriptionModel;
  }

  async create(data) {
    return this.subscriptionModel.create(data);
  }

  async updateByProviderId(
    providerSubscriptionId,
    data
  ) {
    return this.subscriptionModel.findOneAndUpdate(
      {
        providerSubscriptionId,
      },
      data,
      {
        new: true,
      }
    );
  }

  async findByProviderId(
    providerSubscriptionId
  ) {
    return this.subscriptionModel.findOne({
      providerSubscriptionId,
    });
  }
}
```

Repository Responsibility

Repositories ONLY handle database operations.

No business logic. No Stripe logic. No Razorpay logic.

5. Abstract Payment Provider

providers/payment/payment.provider.js

```
export class PaymentProvider {  
  
  async createSubscription(data) {  
    throw new Error("Not implemented");  
  }  
  
  async verifyWebhook(req) {  
    throw new Error("Not implemented");  
  }  
}
```

WHY THIS IS IMPORTANT

This creates a COMMON CONTRACT.

All providers must implement:

```
createSubscription()  
verifyWebhook()
```

6. Stripe Provider

providers/payment/stripe.provider.js

```
import Stripe from "stripe";  
  
import { PaymentProvider }  
from "../payment.provider.js";  
  
export class StripeProvider
```

```

extends PaymentProvider {

  constructor() {
    super();

    this.stripe = new Stripe(
      process.env.STRIPE_SECRET_KEY
    );
  }

  async createSubscription(data) {

    const session =
      await this.stripe.checkout.sessions.create({

        mode: "subscription",

        line_items: [
          {
            price: data.priceId,
            quantity: 1,
          },
        ],

        success_url:
          data.successUrl,

        cancel_url:
          data.cancelUrl,

        metadata: {
          userId: data.userId,
        },
      });

    return {
      provider: "stripe",
      checkoutUrl: session.url,
      externalId: session.id,
    };
  }

  verifyWebhook(req) {

    const signature =
      req.headers["stripe-signature"];

    return this.stripe.webhooks.constructEvent(
      req.body,
      signature,
      process.env.STRIPE_WEBHOOK_SECRET
    );
  }
}

```

```
    );  
  }  
}
```

IMPORTANT THING HERE

Stripe internally creates:

```
Checkout Session  
|  
Subscription
```

But service layer DOES NOT care.

Provider hides that complexity.

7. Razorpay Provider

providers/payment/razorpay.provider.js

```
import Razorpay from "razorpay";  
import crypto from "crypto";  
  
import { PaymentProvider }  
from "../payment.provider.js";  
  
export class RazorpayProvider  
extends PaymentProvider {  
  
  constructor() {  
    super();  
  
    this.razorpay = new Razorpay({  
      key_id: process.env.RAZORPAY_KEY_ID,  
      key_secret: process.env.RAZORPAY_SECRET,  
    });  
  }  
  
  async createSubscription(data) {  
  
    const subscription =  
      await this.razorpay.subscriptions.create({  
  
        plan_id: data.planId,
```

```
        total_count: 12,

        customer_notify: 1,
    });

    return {
        provider: "razorpay",
        subscriptionId: subscription.id,
    };
}

verifyWebhook(req) {

    const body = JSON.stringify(req.body);

    const expectedSignature =
        crypto
            .createHmac(
                "sha256",
                process.env.RAZORPAY_WEBHOOK_SECRET
            )
            .update(body)
            .digest("hex");

    const signature =
        req.headers["x-razorpay-signature"];

    return expectedSignature === signature;
}
```

IMPORTANT

Stripe and Razorpay flows are DIFFERENT.

That is NORMAL.

Provider layer hides provider-specific implementation.

8. Factory Pattern

```
factories/payment-provider.factory.js
```

```
import { StripeProvider }
from "../providers/payment/stripe.provider.js";

import { RazorpayProvider }
from "../providers/payment/razorpay.provider.js";

export class PaymentProviderFactory {

  static create(provider) {

    switch (provider) {

      case "stripe":
        return new StripeProvider();

      case "razorpay":
        return new RazorpayProvider();

      default:
        throw new Error(
          "Invalid provider"
        );
    }
  }
}
```

WHY FACTORY?

Without factory:

```
if stripe...
if razorpay...
```

will spread everywhere.

Factory centralizes provider creation.

9. Service Layer

services/payment.service.js

```
export class PaymentService {
```

```
constructor(
  paymentProvider,
  subscriptionRepository
) {
  this.paymentProvider =
    paymentProvider;

  this.subscriptionRepository =
    subscriptionRepository;
}

async createSubscription(data) {

  const result =
    await this.paymentProvider
      .createSubscription(data);

  await this.subscriptionRepository.create({

    userId: data.userId,

    provider: result.provider,

    providerSubscriptionId:
      result.subscriptionId ||
      result.externalId,

    status: "pending",

    planType: data.planType,

    billingInterval: "monthly",
  });

  return result;
}
```

IMPORTANT SERVICE RESPONSIBILITY

Service layer handles:

- business logic
- orchestration
- validation
- provider-independent workflows

NOT Stripe SDK details.

10. Controller Layer

controllers/payment.controller.js

```
export class PaymentController {

  constructor(paymentService) {
    this.paymentService =
      paymentService;
  }

  createSubscription = async (
    req,
    res
  ) => {

    try {

      const result =
        await this.paymentService
          .createSubscription(req.body);

      res.json(result);

    } catch (error) {

      res.status(500).json({
        message: error.message,
      });
    }
  };
}
```

Controller Responsibility

Controllers should ONLY:

- receive request
- call service
- return response

Controllers should stay THIN.

11. Dependency Injection Setup

routes/payment.routes.js

```
import express from "express";

import { PaymentProviderFactory }
from "../factories/payment-provider.factory.js";

import { PaymentService }
from "../services/payment.service.js";

import { PaymentController }
from "../controllers/payment.controller.js";

import { SubscriptionRepository }
from "../repositories/subscription.repository.js";

import { Subscription }
from "../models/subscription.model.js";

const router = express.Router();

router.post(
  "/subscribe/:provider",

  async (req, res) => {

    const providerName =
      req.params.provider;

    const provider =
      PaymentProviderFactory.create(
        providerName
      );

    const subscriptionRepository =
      new SubscriptionRepository(
        Subscription
      );

    const paymentService =
      new PaymentService(
        provider,
        subscriptionRepository
      );

    const paymentController =
      new PaymentController(
```

```
        paymentService
    );

    return paymentController
        .createSubscription(req, res);
    }
};

export default router;
```

Dependency Injection Flow

```
Route
|
Inject Provider
|
Create Service
|
Create Controller
```

Stripe Request Example

POST /subscribe/stripe

Body:

```
{
  "userId": "123",
  "priceId": "price_123",
  "planType": "pro",
  "successUrl": "http://localhost:5173/success",
  "cancelUrl": "http://localhost:5173/cancel"
}
```

Razorpay Request Example

POST /subscribe/razorpay

Body:

```
{
  "userId": "123",
  "planId": "plan_123",
  "planType": "pro"
}
```

IMPORTANT THING

Notice:

Input payloads differ.

This is NORMAL.

Stripe and Razorpay are DIFFERENT systems.

Do NOT try to force identical provider payloads.

12. Stripe Webhook

webhooks/stripe.webhook.js

```
export const stripeWebhookHandler =
  async (req, res) => {

    try {

      const event =
        stripeProvider.verifyWebhook(req);

      console.log(event.type);

      res.sendStatus(200);

    } catch (error) {

      res.status(400).send(error.message);

    }

  };
```

13. Razorpay Webhook

webhooks/razorpay.webhook.js

```
export const razorpayWebhookHandler =
  async (req, res) => {

    const isValid =
      razorpayProvider.verifyWebhook(req);

    if (!isValid) {
      return res.sendStatus(400);
    }

    res.sendStatus(200);
  };
```

14. app.js

```
import express from "express";
import mongoose from "mongoose";
import dotenv from "dotenv";

import paymentRoutes
  from "./routes/payment.routes.js";

dotenv.config();

const app = express();

app.use(express.json());

app.use("/payments", paymentRoutes);

mongoose.connect(process.env.MONGO_URI)
  .then(() => {
    console.log("MongoDB Connected");
  });

export default app;
```

15. server.js

```
import app from "./app.js";

app.listen(process.env.PORT, () => {
  console.log("Server running");
});
```

FINAL ARCHITECTURE FLOW

Stripe

```
Controller
  |
Service
  |
StripeProvider
  |
Stripe SDK
```

Razorpay

```
Controller
  |
Service
  |
RazorpayProvider
  |
Razorpay SDK
```

IMPORTANT LESSON

Good architecture does NOT mean:

Making Stripe and Razorpay identical.

Good architecture means:

Managing differences cleanly.

Production Improvements

As your project grows, add:

Feature	Why
Queue system	Async webhook processing
Idempotency	Prevent duplicate webhooks
Logger	Monitoring
Event bus	Decouple modules
Transactions	DB consistency
Retry logic	Reliability
Circuit breaker	Provider failure protection
Redis	Caching
Background workers	Heavy processing

MOST IMPORTANT TAKEAWAY

Your application should depend on:

Business abstractions

NOT directly on:

- Stripe SDK
- Razorpay SDK
- provider-specific APIs

This is one of the most important backend architecture principles in scalable systems.